

DEVOPS

Training Documentation

An overview of DevOps culture, practices, and lifecycle ? including a dedicated look at Docker and containerization within the DevOps workflow.

DEV · BUILD · TEST



CI/CD · DOCKER · IaC · MONITORING

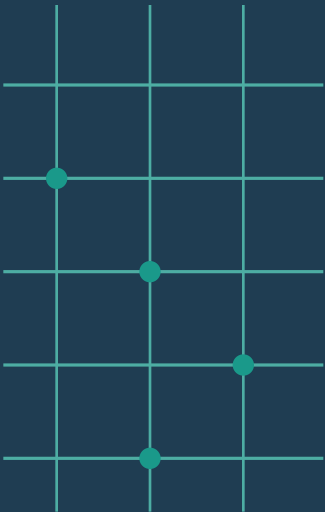


Table of Contents

1. Introduction to DevOps
2. Origin and Evolution of DevOps
3. Core Principles of DevOps
4. The DevOps Lifecycle
5. Continuous Integration and Continuous Delivery (CI/CD)
6. Containerization and Docker
7. Docker Architecture
8. Docker in the DevOps Workflow
9. Container Orchestration
10. Infrastructure as Code (IaC)
11. Monitoring and Logging
12. Popular DevOps Tools
13. Best Practices
14. Summary

1. Introduction to DevOps

1.1 What is DevOps?

- DevOps is a set of practices, culture, and tools that combines software Development (Dev) and IT Operations (Ops).
- It aims to shorten the software development lifecycle while delivering features, fixes, and updates frequently and reliably.
- DevOps is not a single tool or job title — it is a cultural shift built on collaboration, automation, and continuous improvement.

1.2 Why DevOps Matters

- Faster time to market for new features and fixes.
- Improved collaboration between development, operations, and QA teams.
- Higher software quality through automated testing and continuous feedback.
- Reduced deployment failures and faster recovery when issues occur.

1.3 Where DevOps Is Used

Environment	Example Use Case
Cloud-Native Applications	Automated build, test, and deployment pipelines on cloud platforms
Startups	Rapid iteration and scaling with small teams
Enterprise IT	Coordinating large teams and complex release schedules
SaaS Products	Frequent, low-risk releases with continuous monitoring

2. Origin and Evolution of DevOps

2.1 Before DevOps

- Traditionally, development and operations teams worked in silos — developers wrote code, and a separate operations team deployed and maintained it.
- This separation often caused delays, miscommunication, and finger-pointing when releases failed.
- The Agile movement (early 2000s) improved software development practices but did not fully address the gap between development and operations.

2.2 The Birth of DevOps

- The term "DevOps" was popularized by Patrick Debois, who organized the first "DevOpsDays" conference in Belgium in 2009.
- DevOps grew out of the need to apply Agile principles not just to development, but to the entire delivery pipeline, including operations.
- It was influenced by Lean manufacturing principles and Google's Site Reliability Engineering (SRE) practices.

2.3 Timeline at a Glance

Year	Milestone
2001	The Agile Manifesto is published
2007–2008	Growing frustration with the Dev/Ops divide is widely discussed
2009	Patrick Debois organizes the first DevOpsDays conference
2010s	Tools like Jenkins, Docker, and Ansible accelerate DevOps adoption
2013	Docker is released, popularizing containerization
Today	DevOps is a mainstream practice across startups and enterprises alike

3. Core Principles of DevOps

Principle	Description
Collaboration	Breaking down silos between development, operations, and QA teams
Automation	Automating repetitive tasks such as testing, builds, and deployments
Continuous Integration & Delivery	Frequently integrating and releasing small, reliable changes
Monitoring & Feedback	Continuously observing systems and using feedback to improve
Infrastructure as Code	Managing infrastructure through versioned, repeatable code

4. The DevOps Lifecycle

The DevOps lifecycle is often visualized as an infinite loop of continuous phases, reflecting an ongoing cycle of development and operations rather than a one-time process.

Phase	Description
Plan	Define requirements, features, and priorities for the next release
Code	Write and review application source code
Build	Compile code and package it into deployable artifacts
Test	Run automated tests to catch bugs early
Release	Prepare a tested build for deployment
Deploy	Push the release to staging or production environments
Operate	Run and maintain the application in production
Monitor	Track performance, errors, and usage to inform the next Plan phase

5. Continuous Integration and Continuous Delivery (CI/CD)

5.1 Continuous Integration (CI)

- Developers frequently merge code changes into a shared repository.
- Each change automatically triggers a build and a suite of automated tests.
- Catches integration issues early, before they compound into larger problems.

5.2 Continuous Delivery vs. Continuous Deployment

Term	Meaning
Continuous Delivery	Every change that passes automated tests is ready to release, but deployment to production requires a manual approval
Continuous Deployment	Every change that passes automated tests is deployed to production automatically, with no manual step

5.3 A Typical CI/CD Pipeline

Stage	Purpose
Source	Pull the latest code from version control (e.g. Git)
Build	Compile the application and its dependencies
Test	Run unit, integration, and other automated tests
Package	Create a deployable artifact, often a Docker image
Deploy	Release the artifact to a staging or production environment

```
# Simplified CI/CD pipeline (YAML-style)
stages:
  - build
  - test
  - deploy

build:
  script: docker build -t myapp:latest .

test:
  script: docker run myapp:latest npm test

deploy:
  script: docker push myapp:latest
```

6. Containerization and Docker

6.1 What Is Containerization?

- Containerization packages an application together with everything it needs to run — code, runtime, libraries, and configuration — into a single, isolated unit called a container.
- Containers share the host machine's operating system kernel, making them far lighter and faster to start than traditional virtual machines.
- This ensures an application behaves the same way on a developer's laptop, in testing, and in production.

6.2 What Is Docker?

- Docker is the most widely used open-source platform for building, shipping, and running containerized applications.
- It was created by Solomon Hykes and first released publicly in March 2013 by the company dotCloud (later renamed Docker, Inc.).

- Docker standardized containers and made them accessible with simple commands and a rich ecosystem of tools.

6.3 Why Docker Matters in DevOps

Benefit	Explanation
Consistency	Solves the "it works on my machine" problem by packaging the exact same environment everywhere
Speed	Containers start in seconds, far faster than booting a virtual machine
Portability	A Docker image runs the same way on a laptop, a test server, or in the cloud
CI/CD Integration	Applications can be built once as an image and deployed unchanged through every pipeline stage
Resource Efficiency	Multiple containers can share a host's OS kernel, using far less memory and disk than separate VMs

6.4 Docker vs. Virtual Machines

Aspect	Virtual Machine	Docker Container
Operating System	Each VM includes a full guest OS	Containers share the host OS kernel
Boot Time	Minutes	Seconds
Size	Often gigabytes	Typically megabytes
Resource Usage	Higher — dedicated OS overhead per VM	Lower — lightweight, shared kernel
Isolation	Strong, hardware-level isolation	Process-level isolation, still very effective

7. Docker Architecture

7.1 Client-Server Model

Component	Description
Docker Client	The command-line tool (docker) that users interact with to issue commands
Docker Daemon (dockerd)	A background service that builds, runs, and manages containers
Docker Images	Read-only templates used to create containers
Docker Containers	Running instances of images, isolated from each other and the host
Docker Registry	A storage and distribution system for images (e.g. Docker Hub)

7.2 Underlying Linux Technologies

- Namespaces — isolate a container's view of processes, networking, and file systems from the host and other containers.
- Control Groups (cgroups) — limit and monitor how much CPU, memory, and other resources a container can use.
- Union File Systems — allow images to be built in stackable, reusable layers, saving disk space and build time.

7.3 The Dockerfile

A Dockerfile is a text file containing step-by-step instructions for building a Docker image automatically.

```
# Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

7.4 Common Dockerfile Instructions

Instruction	Purpose
FROM	Specifies the base image to build from

Instruction	Purpose
WORKDIR	Sets the working directory inside the container
COPY / ADD	Copies files from the host into the image
RUN	Executes a command while building the image (e.g. installing packages)
EXPOSE	Documents which port the container listens on
CMD / ENTRYPOINT	Defines the command that runs when the container starts

8. Docker in the DevOps Workflow

8.1 Basic Docker Commands

Command	Purpose
docker build -t name .	Build an image from a Dockerfile in the current directory
docker images	List locally available images
docker run -d -p 8080:80 name	Run a container in the background, mapping a host port to a container port
docker ps	List currently running containers
docker stop id	Stop a running container
docker exec -it id bash	Open an interactive shell inside a running container
docker logs id	View the output/logs of a container

8.2 Docker in a CI/CD Pipeline

- Build — the pipeline builds a Docker image from the application's Dockerfile.
- Test — automated tests run inside a container built from that same image.
- Push — the tested image is pushed to a registry such as Docker Hub or a private registry.
- Deploy — the exact same image is pulled and run in staging and production, guaranteeing consistency.

8.3 Docker Compose

Docker Compose lets developers define and run multi-container applications using a single YAML file, useful for local development environments that combine, for example, an application and a database.

```
# docker-compose.yml
version: "3.9"
services:
  web:
    build: .
    ports:
      - "3000:3000"
  database:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: example
```

```
$ docker-compose up -d
$ docker-compose down
```

9. Container Orchestration

9.1 Why Orchestration Is Needed

- Running a handful of containers manually is manageable, but production systems often need hundreds of containers scaled across many machines.
- Orchestration tools automate deployment, scaling, networking, and recovery of containers.

9.2 Kubernetes Overview

Concept	Description
Pod	The smallest deployable unit, wrapping one or more containers
Node	A physical or virtual machine that runs pods
Cluster	A set of nodes managed together by Kubernetes
Service	A stable network endpoint that exposes a set of pods
Deployment	Manages the desired state and scaling of a set of pods

9.3 Docker Swarm

Docker Swarm is Docker's own, simpler built-in orchestration tool. It clusters multiple Docker hosts and schedules containers across them, offering an easier learning curve than Kubernetes for smaller-scale deployments.

10. Infrastructure as Code (IaC)

10.1 What Is IaC?

- Infrastructure as Code manages and provisions servers, networks, and other infrastructure through machine-readable configuration files, instead of manual setup.
- Infrastructure changes become versioned, reviewable, and repeatable, just like application code.

10.2 Benefits of IaC

- Consistency — the same configuration produces identical environments every time.
- Speed — new environments can be provisioned in minutes rather than days.
- Version Control — infrastructure changes can be tracked, reviewed, and rolled back.

10.3 Popular IaC Tools

Tool	Purpose
Terraform	Provisions and manages cloud infrastructure across many providers
Ansible	Automates configuration management and application deployment
AWS CloudFormation	Provisions AWS resources using declarative templates
Pulumi	Defines infrastructure using general-purpose programming languages

```
# Simplified Terraform example
resource "aws_instance" "web" {
  ami          = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
}
```

11. Monitoring and Logging

11.1 Why Monitoring Matters

- Monitoring provides visibility into how applications and infrastructure are performing in real time.
- Early detection of issues reduces downtime and helps teams respond before problems affect users.

11.2 Key Metrics to Track

Metric	Why It Matters
Uptime / Availability	Measures how reliably a service is accessible to users
Latency	Tracks how quickly requests are processed
Error Rate	Highlights failures that may need immediate attention
Resource Usage	CPU, memory, and disk usage help with capacity planning

11.3 Popular Monitoring Tools

Tool	Purpose
Prometheus	Collects and stores time-series metrics from applications and infrastructure
Grafana	Visualizes metrics through customizable dashboards
ELK Stack	Collects, searches, and visualizes log data (Elasticsearch, Logstash, Kibana)
Datadog	A commercial, all-in-one monitoring and analytics platform

12. Popular DevOps Tools

Category	Example Tools
Version Control	Git, GitHub, GitLab, Bitbucket
CI/CD	Jenkins, GitHub Actions, GitLab CI, CircleCI
Containerization	Docker, Podman
Orchestration	Kubernetes, Docker Swarm
Configuration Management	Ansible, Chef, Puppet
Infrastructure as Code	Terraform, Pulumi, CloudFormation
Monitoring & Logging	Prometheus, Grafana, ELK Stack, Datadog

13. Best Practices

- Automate everything that is repeated more than once — builds, tests, deployments, and infrastructure changes.
- Keep deployments small and frequent, rather than large and infrequent, to reduce risk.
- Use version control for both application code and infrastructure configuration.
- Build small, focused Docker images using official base images and multi-stage builds to reduce size and attack surface.
- Never run containers as the root user in production; apply the principle of least privilege.
- Monitor systems continuously and treat alerts and incidents as opportunities to improve, not to assign blame.
- Document processes and encourage knowledge-sharing across development and operations teams.

14. Summary

DevOps brings together the practices, culture, and tools needed to deliver software faster, more reliably, and with better collaboration between development and operations. This documentation has covered its origins and core principles, the continuous DevOps lifecycle, and CI/CD pipelines that automate building, testing, and releasing software. It gave particular attention to Docker and containerization — covering what Docker is, its architecture, the Dockerfile, essential commands, and how Docker fits naturally into CI/CD pipelines through Docker Compose. It closed with an overview of container orchestration, Infrastructure as Code, monitoring, and the broader DevOps tooling ecosystem.

Key Takeaway

Docker is one of the most important tools in modern DevOps because it makes environments consistent, portable, and fast to deploy — but DevOps itself is a broader culture of collaboration and automation that Docker helps support, not a replacement for it.